

Module 14 - chapter 3

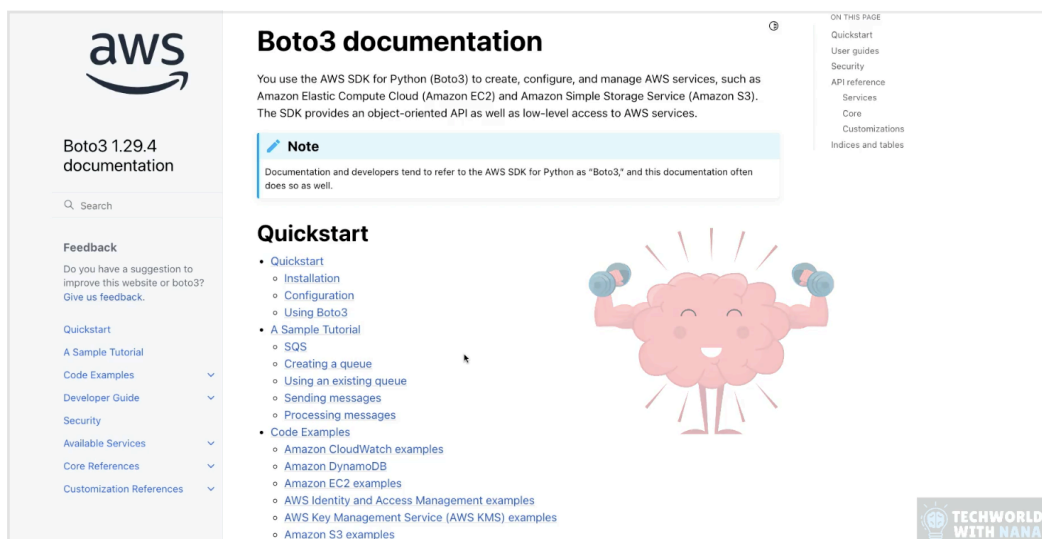
Getting familiar with Boto

Let's test boto3 and make a call to AWS to return a list of the VPCs in the region we configured which is eu-west-2

-> cat ~/.aws/config

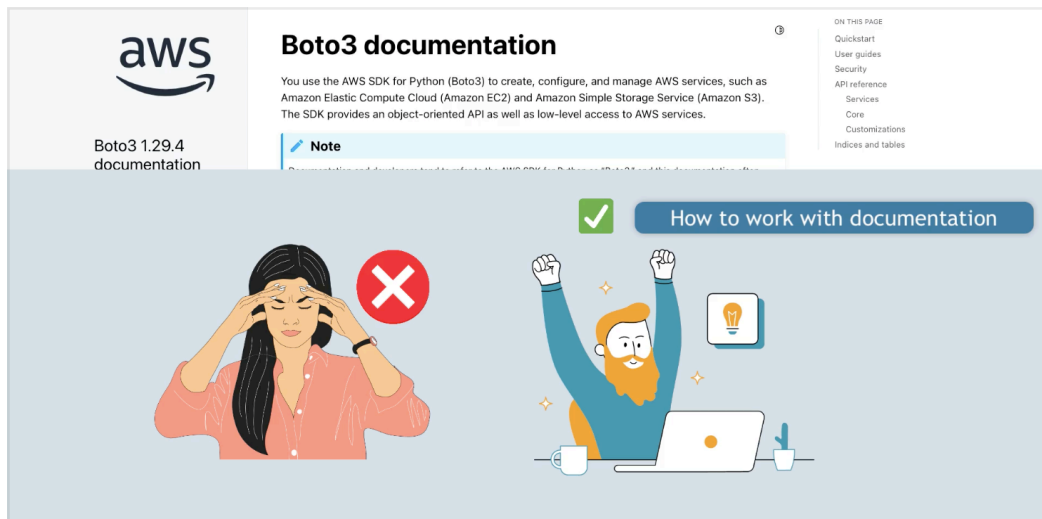
```
> cat ~/.aws/config
[default]
region = eu-west-2
output = json
```

Now let's look at boto3 documentation. Boto3 can do much more than TF can do.



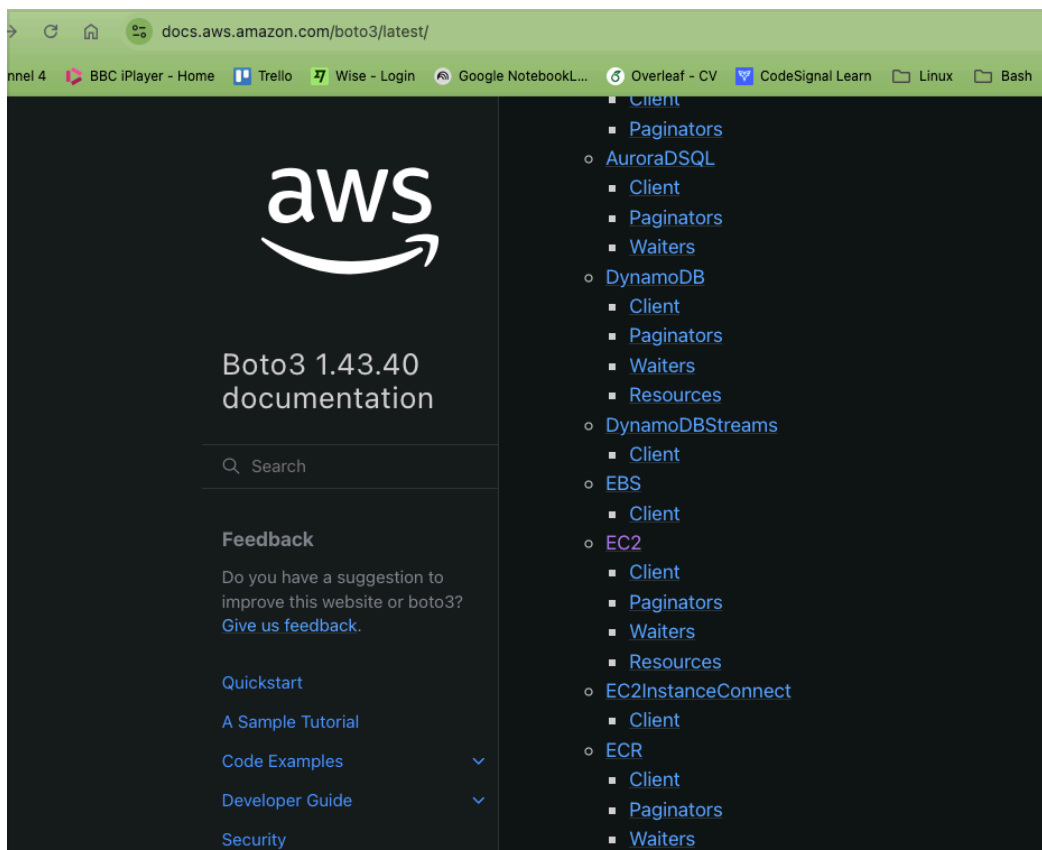
The screenshot shows the AWS Boto3 documentation page. On the left, there's a sidebar with the AWS logo and navigation links for 'Boto3 1.29.4 documentation', 'Feedback', 'Quickstart', 'A Sample Tutorial', 'Code Examples', 'Developer Guide', 'Security', 'Available Services', 'Core References', and 'Customization References'. The main content area is titled 'Boto3 documentation' and includes a 'Note' box stating that documentation and developers often refer to the AWS SDK for Python as 'Boto3'. Below this is a 'Quickstart' section with a list of links: 'Quickstart', 'Installation', 'Configuration', 'Using Boto3', 'A Sample Tutorial', 'SQS', 'Creating a queue', 'Using an existing queue', 'Sending messages', 'Processing messages', 'Code Examples', 'Amazon CloudWatch examples', 'Amazon DynamoDB', 'Amazon EC2 examples', 'AWS Identity and Access Management examples', 'AWS Key Management Service (AWS KMS) examples', and 'Amazon S3 examples'. A cartoon brain character is positioned to the right of the 'Quickstart' section. In the bottom right corner, there's a 'TECHWORLD WITH NANA' logo.

We need to understand how boto3 documentation works not to memorise it:



So we first create a python or boto3 client. The client will connect to AWS

And for EC2



Here is an example of how to create a client

The screenshot shows the AWS Boto3 1.43.40 documentation page for the EC2 Client. The left sidebar contains the AWS logo, the version number, a search bar, and a navigation menu with links to Feedback, Quickstart, A Sample Tutorial, Code Examples, Developer Guide, Security, and Available Services. The main content area is titled 'EC2 Client' and describes the `EC2.Client` class as a low-level client for the Amazon Elastic Compute Cloud (EC2) service. It provides a list of links to learn more about the Query API, permissions, API actions, and error codes. Below this, it lists alternative methods to access the Amazon EC2 API, including the AWS CLI, CloudFormation, PowerShell, and SDKs. A code block shows the Python code to create an EC2 client:

```
import boto3
client = boto3.client('ec2')
```

. The page also lists available methods for the client, such as `accept_address_transfer`.

Q:

what a client is? why I need to create an EC2 **client** to access the list of VPCs instead of a VPC client?

A:

What a "client" is: In boto3, a client is a low-level object that represents a connection to a specific AWS service's API. When you do `boto3.client('ec2')`, you get an object with methods that map directly to that service's API calls (e.g. `describe_vpcs`, `describe_instances`, `run_instances`). It handles auth, request signing, retries, and talks over HTTPS to AWS's endpoints for you.

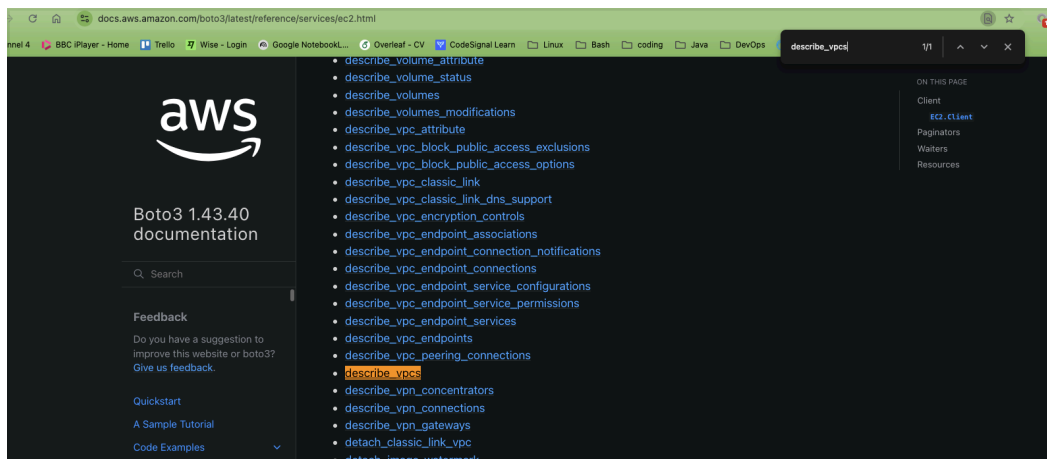
Why there's no "VPC client": AWS doesn't organize its API by individual resource type — it organizes it by service. VPCs, subnets, route tables, internet gateways, security groups, NAT gateways, etc. are all resources that live under the EC2 service in AWS's own API design (this is a historical/organizational choice — VPC functionality was built as part of EC2's networking layer). So there's an `ec2` client, and VPC-related calls (`describe_vpcs`, `create_vpc`, `delete_vpc`, etc.) are just a subset of the methods available on that one client — same as `describe_instances` or `describe_security_groups` are.

So the mental model is:

- boto3 client granularity = AWS service (`ec2`, `s3`, `iam`, `lambda`, `rds`...)
- Not resource type (`vpc`, `subnet`, `bucket`...)

```
main.py x
1 import boto3
2
3 ec2_client = boto3.client('ec2')
```

And to list vpcs we use -> describe_vpcs



describe_vpcs

`EC2.Client.describe_vpcs(**kwargs)`

Describes your VPCs. The default is to describe all your VPCs. Alternatively, you can specify specific VPC IDs or filter the results to include only the VPCs that match specific criteria.

See also: [AWS API Documentation](#)

Request Syntax

```
response = client.describe_vpcs(
    Filters=[
        {
            'Name': 'string',
            'Values': [
                'string',
            ]
        },
    ],
    VpcIds=[
        'string',
    ],
    NextToken='string',
    MaxResults=123,
    DryRun=True|False
)
```

What do we need to know?

What do we need to know?



- the function name, we need to call
- which parameters we need to pass?
- what is returned from the function?

- The function name we need to call -> **describe_vpcs()**

```

1 import boto3
2
3 ec2_client = boto3.client('ec2')
4
5 ec2_client.describe_vpcs()

```

– What parameters we need to pass?

PARAMETERS:

- **Filters (list)** – The filters.
 - `cidr` – The primary IPv4 CIDR block of the VPC. The CIDR block you specify must exactly match the VPC's CIDR block for information to be returned for the VPC. Must contain the slash followed by one or two digits (for example, `/28`).
 - `cidr-block-association.cidr-block` – An IPv4 CIDR block associated with the VPC.
 - `cidr-block-association.association-id` – The association ID for an IPv4 CIDR block associated with the VPC.
 - `cidr-block-association.state` – The state of an IPv4 CIDR block associated with the VPC.
 - `dhcp-options-id` – The ID of a set of DHCP options.
 - `ipv6-cidr-block-association.ipv6-cidr-block` – An IPv6 CIDR block associated with the VPC.
 - `ipv6-cidr-block-association.ipv6-pool` – The ID of the IPv6 address pool from which the IPv6 CIDR block is allocated.
 - `ipv6-cidr-block-association.association-id` – The association ID for an IPv6 CIDR block associated with the VPC.
 - `ipv6-cidr-block-association.state` – The state of an IPv6 CIDR block associated with the VPC.
 - `is-default` – Indicates whether the VPC is the default VPC.
 - `owner-id` – The ID of the Amazon Web Services account that owns the VPC.
 - `state` – The state of the VPC (`pending` | `available`).
 - `tag` – The key/value combination of a tag assigned to the resource. Use the tag key in the filter name and the tag value as the filter value. For example, to find all resources that have a tag with the key `Owner` and the value `TeamA`, specify `tag:Owner` for the filter name and `TeamA` for the filter value.
 - `tag-key` – The key of a tag assigned to the resource. Use this filter to find all resources assigned a tag with a specific key, regardless of the tag value.
 - `vpc-id` – The ID of the VPC.

For this function the parameters are optional. So if we don't set them we will just get the complete list of vpcs

`EC2.Client.describe_vpcs(**kwargs)`

Describes your VPCs. The default is to describe all your VPCs. Alternatively, you can specify specific VPC IDs or filter the results to include only the VPCs that match specific criteria.

See also: [AWS API Documentation](#)

Documentation also gives us the response Syntax:

- what the function returns

It returns a dictionary '{' and as for the Vpcs it will be a list '['

Response Syntax

```
{
  'NextToken': 'string',
  'Vpcs': [
    {
      'OwnerId': 'string',
      'InstanceTenancy': 'default'|'dedicated'|'host',
      'Ipv6CidrBlockAssociationSet': [
        {
          'AssociationId': 'string',
          'Ipv6CidrBlock': 'string',
          'Ipv6CidrBlockState': {
            'State': 'associating'|'associated'|'disassociating'|'disass',
            'StatusMessage': 'string'
          },
          'NetworkBorderGroup': 'string',
          'Ipv6Pool': 'string',
          'Ipv6AddressAttribute': 'public'|'private',
          'IpSource': 'amazon'|'byoip'|'none'
        },
      ],
      'CidrBlockAssociationSet': [
        {
```

So we have this code:

```
1 import boto3
2
3 ec2_client = boto3.client('ec2')
4
5 all_available_vpcs = ec2_client.describe_vpcs()
6 print(all_available_vpcs)
```

Response

```
↑ /Users/astrid/PythonProject/Automation_with_python/.venv/bin/p
↓ {'Vpcs': [{ 'OwnerId': '706976333940', 'InstanceTenancy': 'defa
⇌
⇓ Process finished with exit code 0
```

We usually just need a specific information like the VpcId or the state or the CidrBlock information, so we need to get used to call and navigate the nested response.

So let's practice to get the VpcId and the CidrBlockState

Response Syntax

```
{
  'NextToken': 'string',
  'Vpcs': [
    {
      'OwnerId': 'string',
      'InstanceTenancy': 'default'|'dedicated'|'host',
      'Ipv6CidrBlockAssociationSet': [
        {
          'AssociationId': 'string',
          'Ipv6CidrBlock': 'string',
          'Ipv6CidrBlockState': {
            'State': 'associating'|'associated'|'disassociating'|'disass
            'StatusMessage': 'string'
          },
          'NetworkBorderGroup': 'string',
          'Ipv6Pool': 'string',
          'Ipv6AddressAttribute': 'public'|'private',
          'IpSource': 'amazon'|'byoip'|'none'
        },
      ],
      'CidrBlockAssociationSet': [
        {
          'AssociationId': 'string',
          'CidrBlock': 'string',
          'CidrBlockState': {
            'State': 'associating'|'associated'|'disassociating'|'disass
            'StatusMessage': 'string'
          }
        },
      ],
    },
  ],
}
```



```

        'BlockPublicAccessStates': {
            'InternetGatewayBlockMode': 'off'|'block-bidire
        },
        'VpcId': 'string',
        'State': 'pending'|'available',
        'CidrBlock': 'string',
        'DhcpOptionsId': 'string'
    },
]
}

```

Response Structure

And the code to get the VpcId for each VPC dictionary in the list of vpcs:

```

main.py x
1  import boto3
2
3  ec2_client = boto3.client('ec2')
4
5  all_available_vpcs = ec2_client.describe_vpcs()
6  vpcs = all_available_vpcs["Vpcs"]
7
8  # loop the list of vpcs which is a list of dictionaries
9  for vpc in vpcs:
10     # in a dictionary I get a value using its key name
11     print(vpc["VpcId"])  You, Moments ago • Uncommitted changes
12
Run  main x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Use
vpc-03677c88f7cc2f715
Process finished with exit code 0

```

Now let's get 'CidrBlockState' but this is nested inside 'CidrBlockAssociationSet' so let's first access 'CidrBlockAssociationSet'

```
main.py x
1 import boto3
2
3 ec2_client = boto3.client('ec2')
4
5 all_available_vpcs = ec2_client.describe_vpcs()
6 vpcs = all_available_vpcs["Vpcs"]
7
8 # loop the list of Vpcs which is a list of dictionaries
9 for vpc in vpcs:
10     # in a dictionary I get a value using its key name
11     print(vpc["VpcId"])
12     print(vpc["CidrBlockAssociationSet"]) You, 2 minutes ago • Uncommitted changes
13
Run main x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Autom
vpc-03677c88f7cc2f715
{'AssociationId': 'vpc-cidr-assoc-0c88b9ed9e608d8a2', 'CidrBlock': '172.31.0.0/16', 'CidrBlockState'
Process finished with exit code 0
```

And we can see that it returns a list of dictionaries, so we have to iterate this list as well:

```
8 # loop the list of Vpcs which is a list of dictionaries
9 for vpc in vpcs:
10     # in a dictionary I get a value using its key name
11     print(vpc["VpcId"])
12     cidr_block_assoc_sets = vpc["CidrBlockAssociationSet"]
13     # iterate over the list
14     for assoc_set in cidr_block_assoc_sets:
15         print(assoc_set) You, 4 minutes ago • Uncommitted changes
16
17
Run main x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/Python
vpc-03677c88f7cc2f715
{'AssociationId': 'vpc-cidr-assoc-0c88b9ed9e608d8a2', 'CidrBlock': '172.31.0.0/16', 'Cid
Process finished with exit code 0
```

Now we are getting closer.

Let's get the 'CidrBlockState' from each dictionary

```
main.py x
3 ec2_client = boto3.client('ec2')
4
5 all_available_vpcs = ec2_client.describe_vpcs()
6 vpcs = all_available_vpcs["Vpcs"]
7
8 # loop the list of Vpcs which is a list of dictionaries
9 for vpc in vpcs:
10     # in a dictionary I get a value using its key name
11     print(vpc["VpcId"])
12     cidr_block_assoc_sets = vpc["CidrBlockAssociationSet"]
13     # iterate over the list
14     for assoc_set in cidr_block_assoc_sets:
15         print(assoc_set["CidrBlockState"]) You, 41 minutes ago •
16
Run main x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /
vpc-03677c88f7cc2f715
{'State': 'associated'}
Process finished with exit code 0
```

And we got the 'state' attribute (the object doesn't have a 'StatusMessage' attribute and this is the reason is not printed)

```
    'CidrBlockAssociationSet': [
        {
            'AssociationId': 'string',
            'CidrBlock': 'string',
            'CidrBlockState': {
                'State': 'associating'|'associated'|'disa
                'StatusMessage': 'string'
            }
        },
    ],
```

Connect to non-default region

What if I want to access a different region without changing the AWS configure region?

```
> cat ~/.aws/config
[default]
region = eu-west-2
output = json
```

So I want to change the region only for this python project for that we override the default region when creating the client

And we pass an argument to override 'region_name'

```
1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-3')
```

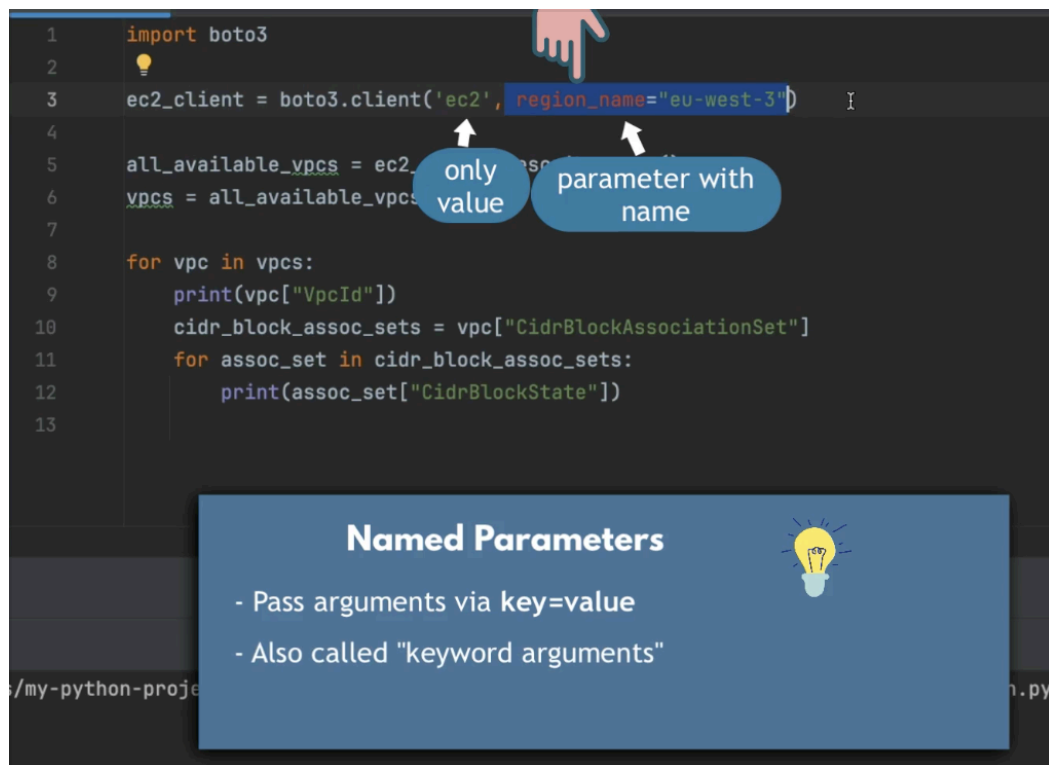
And we get a different Vpc id

```
main.py x
1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-3')
4
5 all_available_vpcs = ec2_client.describe_vpcs()
6 vpcs = all_available_vpcs["Vpcs"]
7
8 # loop the list of Vpcs which is a list of dictionaries
9 for vpc in vpcs:
10     # in a dictionary I get a value using its key name
11     print(vpc["VpcId"])
12     cidr_block_assoc_sets = vpc["CidrBlockAssociationSet"]
13     # iterate over the list
14     for assoc_set in cidr_block_assoc_sets:
```

```
Run main x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automation_with_python/main.py
vpc-p424e806da8ab5368
{'State': 'associated'}
```

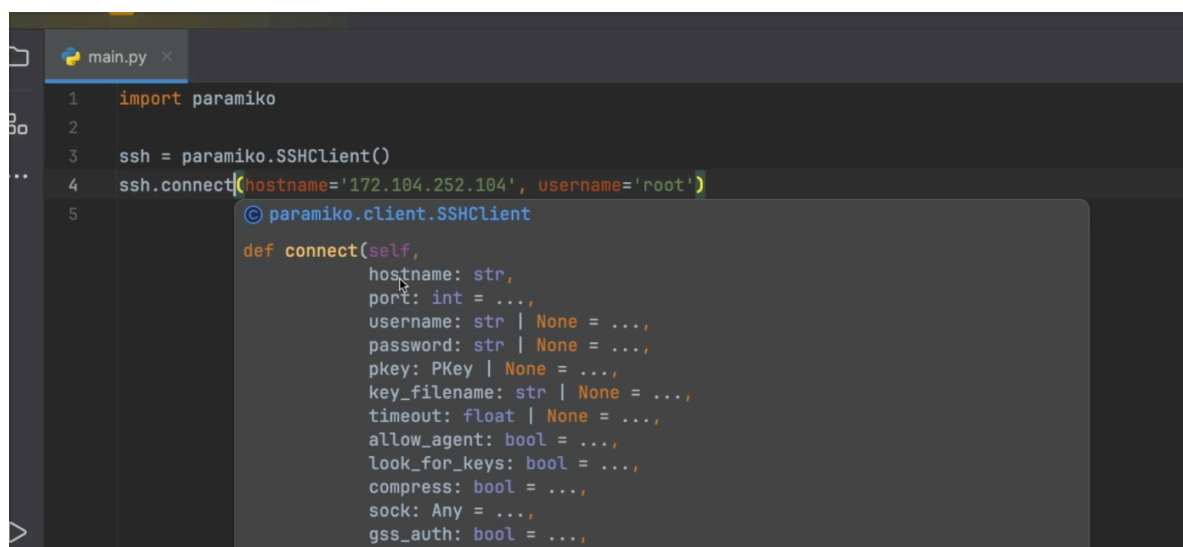
Process finished with exit code 0

To override the region we pass the parameter name along with the value of the region as an argument



This type of parameters/arguments are called 'Named parameters' or 'keyword arguments'

An example below where the function has many optional parameters:



So if we only need few of the parameters we need to tell python which parameters we are passing.

If we pass arguments in the same order that parameters are defined in the function then I don't need to let python know the name of the parameters as it will match the first argument with the first parameter, the second argument with the second parameter, etc.

So this is the same:

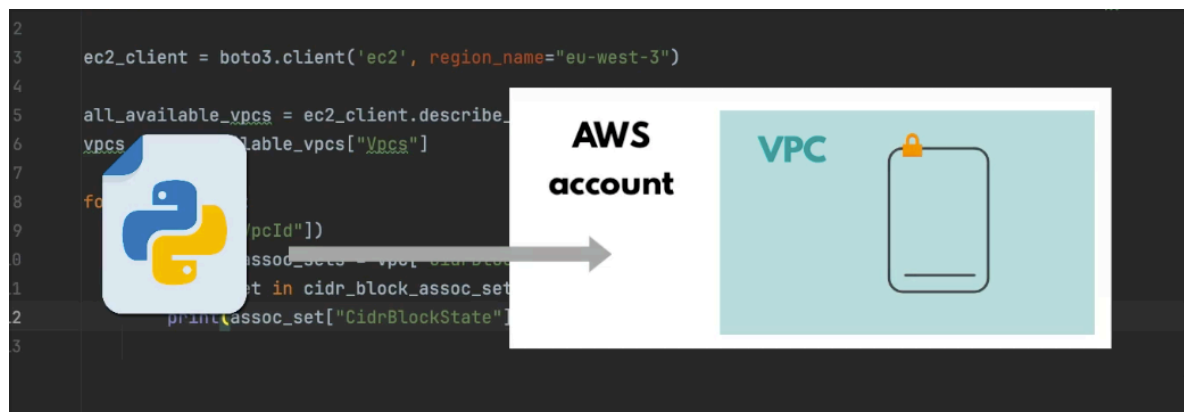
```
4 ssh.connect(hostname='172.104.252.104', port=22, username='root')
```

As this:

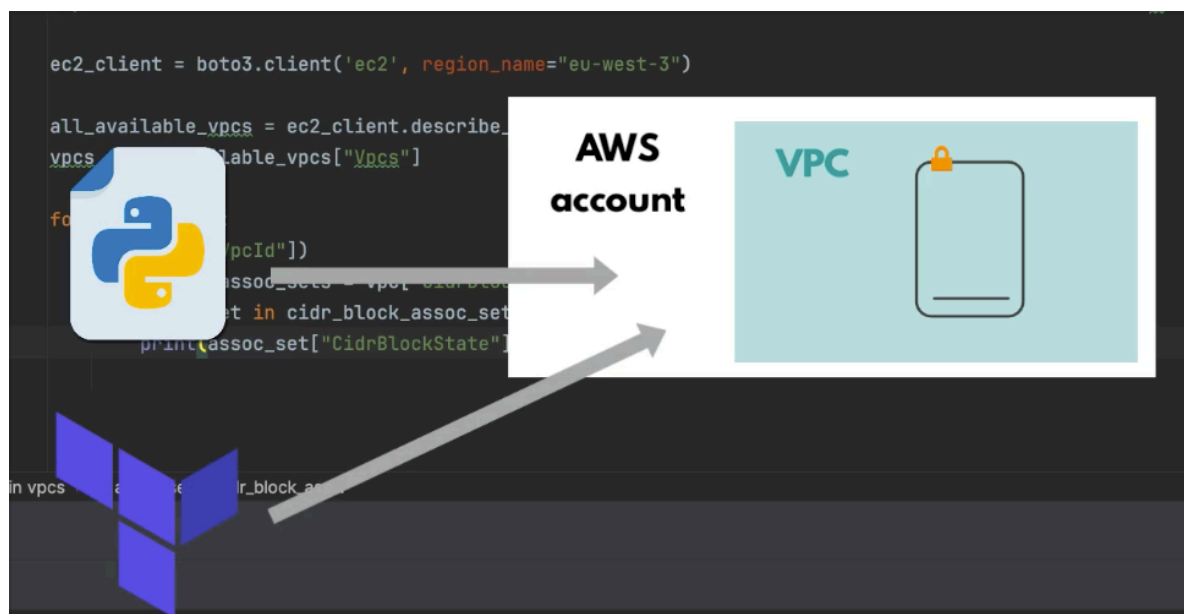
```
4 ssh.connect('172.104.252.104', 22, 'root')
```

Because in the function definition the order is: hostname -> port -> username

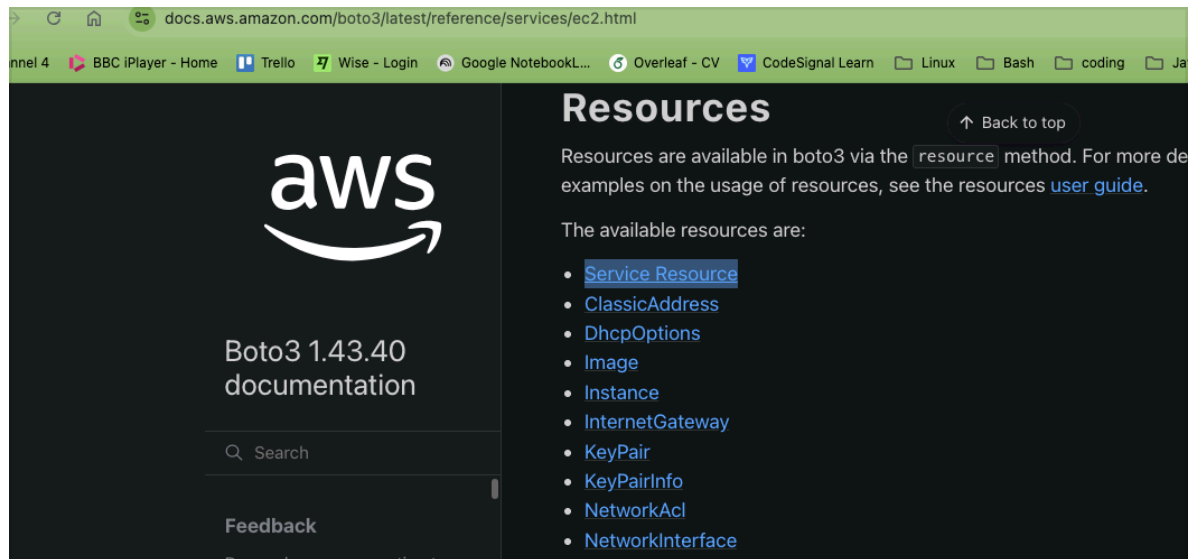
Create VPC and Subnets



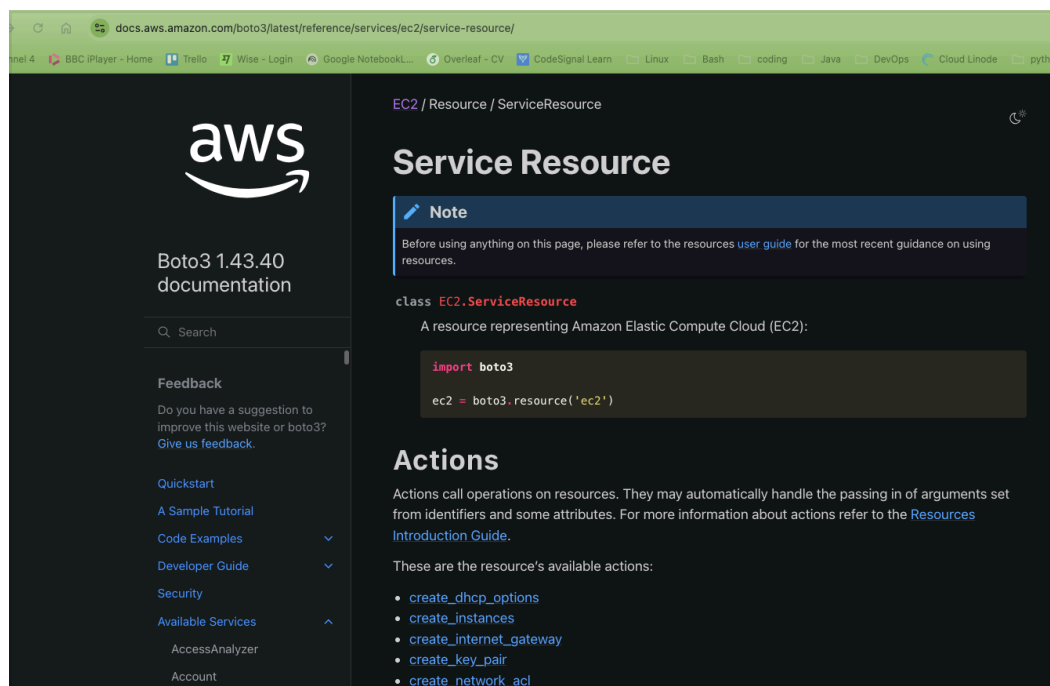
Then we will compare it with how we did it with TF



So we go to the boto3 documentation -> ec2 -> resources -> service resource
<https://docs.aws.amazon.com/boto3/latest/reference/services/ec2.html>



<https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/service-resource/>



Documentation tells us to use
-> .resource()

Instead of
-> .client()

client vs resource



Client = more low-level API

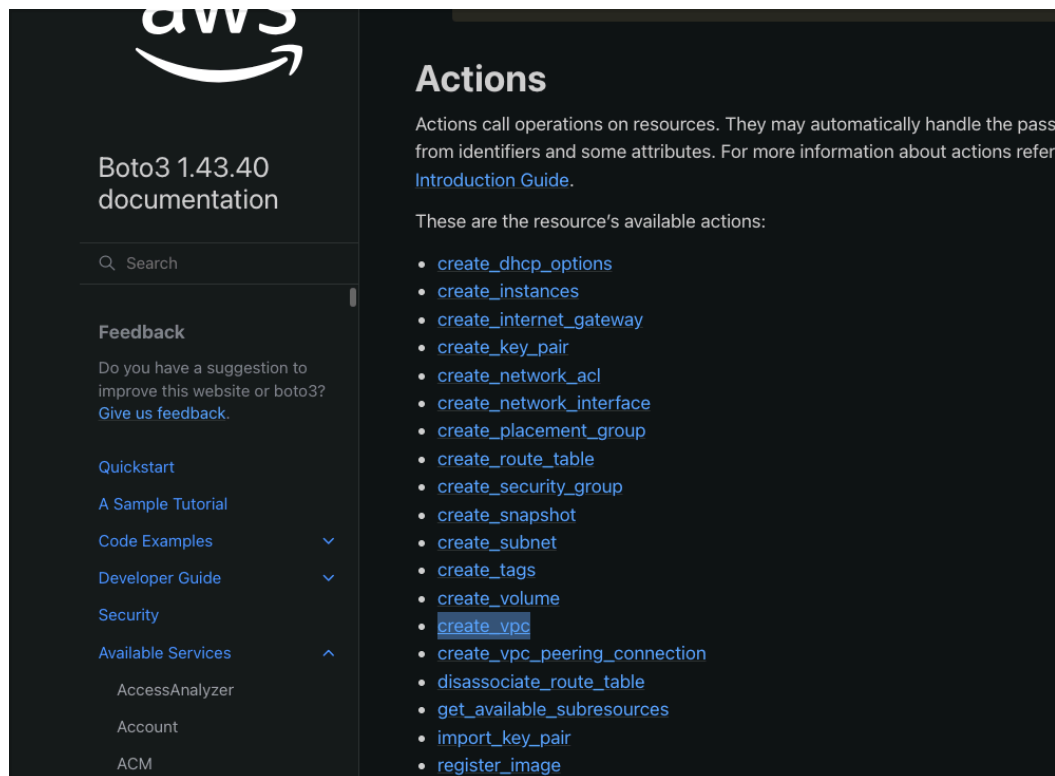
Client = provides one-to-one mapping
to underlying HTTP API operations

Resource = provides resource objects
to access attributes and perform actions

Resource = high-level and object-oriented

Client -> low level where I can access to anything related to EC2 in the API
Resource -> is like a wrapper on top of the client() to access only the resources of the client, it is higher level. So if we want to access an EC2 resource we can have a shortcut by just using `.resource('ec2')` instead of having to navigate the client.

So we can see that one of the available functions for the EC2 resources is `create_vpc`



Boto3 1.43.40 documentation

Q Search

Feedback

Do you have a suggestion to improve this website or boto3? Give us feedback.

Quickstart

A Sample Tutorial

Code Examples

Developer Guide

Security

Available Services

AccessAnalyzer

Account

ACM

Actions

Actions call operations on resources. They may automatically handle the pass from identifiers and some attributes. For more information about actions refer [Introduction Guide](#).

These are the resource's available actions:

- [create_dhcp_options](#)
- [create_instances](#)
- [create_internet_gateway](#)
- [create_key_pair](#)
- [create_network_acl](#)
- [create_network_interface](#)
- [create_placement_group](#)
- [create_route_table](#)
- [create_security_group](#)
- [create_snapshot](#)
- [create_subnet](#)
- [create_tags](#)
- [create_volume](#)
- [create_vpc](#)
- [create_vpc_peering_connection](#)
- [disassociate_route_table](#)
- [get_available_subresources](#)
- [import_key_pair](#)
- [register_image](#)

And under the hood `.resources()` is using `.client()`

So we create our resource

```
main.py x
1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-3')
4 ec2_resource = boto3.resource(*args: 'ec2', region_name='eu-west-3')
5
```

And now we create the vpc so for that we need to see the docs for `create_vpc`

The screenshot shows the AWS Boto3 documentation page for the `create_vpc` action. The left sidebar contains the AWS logo, the version 'Boto3 1.43.40 documentation', a search bar, and a navigation menu with links for Feedback, Quickstart, A Sample Tutorial, Code Examples, Developer Guide, Security, Available Services, AccessAnalyzer, and Account. The main content area is titled 'create_vpc' and includes the following text:

ServiceResource / Action / create_vpc

EC2.ServiceResource.create_vpc(kwargs)**

Creates a VPC with the specified CIDR blocks.

A VPC **must** have an associated IPv4 CIDR block. You can choose an IPv4 CIDR block or an IPAM-allocated IPv4 CIDR block. You can optionally associate an IPv6 CIDR block with a VPC. You can choose an IPv6 CIDR block, an Amazon-provided IPv6 CIDR block, an IPAM-allocated IPv6 CIDR block, or an IPv6 CIDR block that you brought to Amazon Web Services. For more information, see [IP addressing for your VPCs and subnets](#) in the *Amazon VPC User Guide*.

By default, each instance that you launch in the VPC has the default DHCP options, which include only a default DNS server that we provide (AmazonProvidedDNS). For more information, see [DHCP option sets](#) in the *Amazon VPC User Guide*.

You can specify DNS options and tenancy for a VPC when you create it. You can't change the tenancy of a VPC after you create it. For more information, see [VPC configuration options](#) in the *Amazon VPC User Guide*.

See also: [AWS API Documentation](#)

Request Syntax

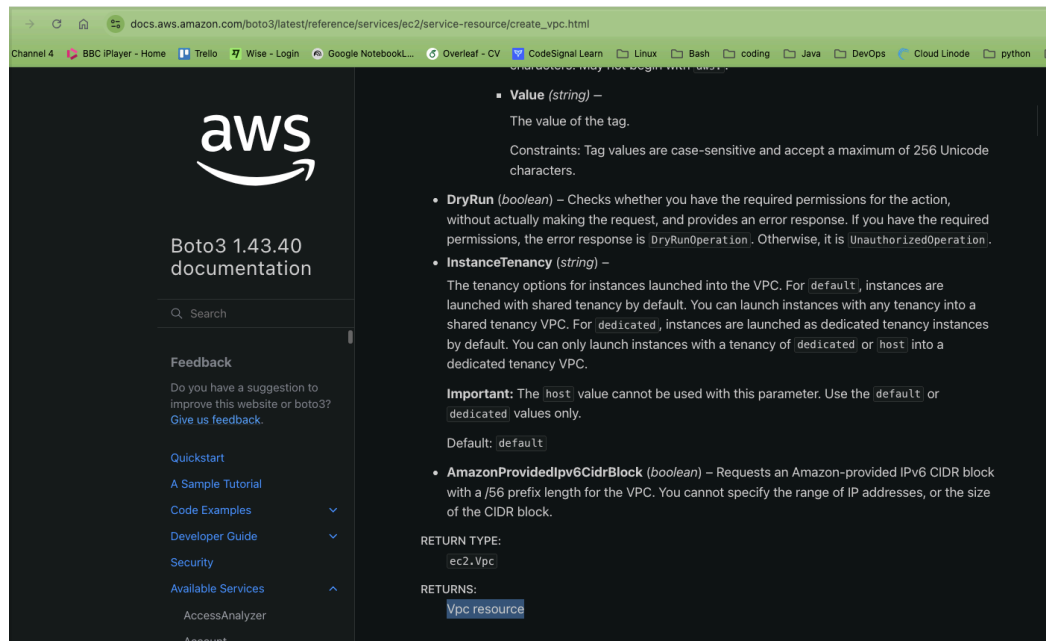
```
vpc = ec2.create_vpc(
    CidrBlock='string',
    Ipv6Pool='string',
    Ipv6CidrBlock='string',
    Ipv4IpamPoolId='string',
    Ipv4NetmaskLength=123,
    Ipv6IpamPoolId='string'
```

"A VPC must have an associated IPv4 CIDR block" And it says that one parameter is mandatory -> **CidrBlock**

When working with terraform we had to specify the cidr block when creating a VPC and here is the same.

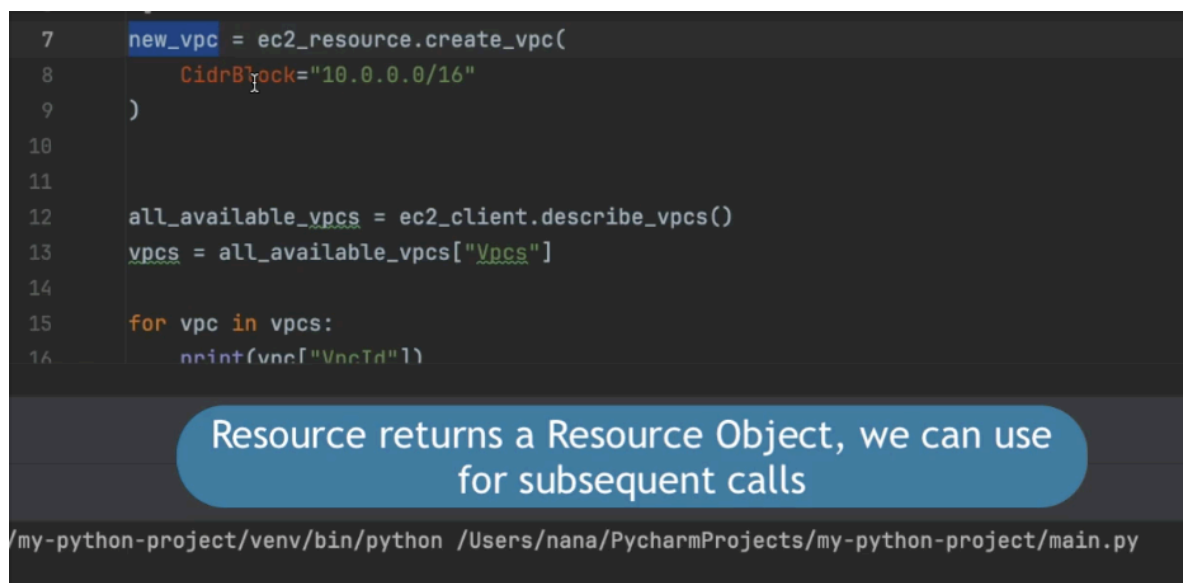
```
1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-3')
4 ec2_resource = boto3.resource(*args: 'ec2', region_name='eu-west-3')
5
6 ec2_resource.create_vpc(
7     CidrBlock='10.0.0.0/16',
8 )
9
```

And the docs tell us that `.create_vpc()` returns Vpc resource

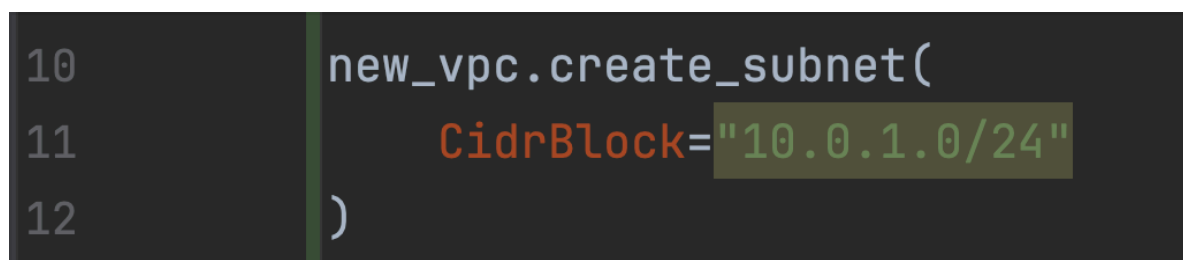


So we get a Resource object, of type vpc, back that then we can reuse.

And we can store the response in a variable like 'new_vpc'



Now we can call the vpc and create a subnet for that vpc:

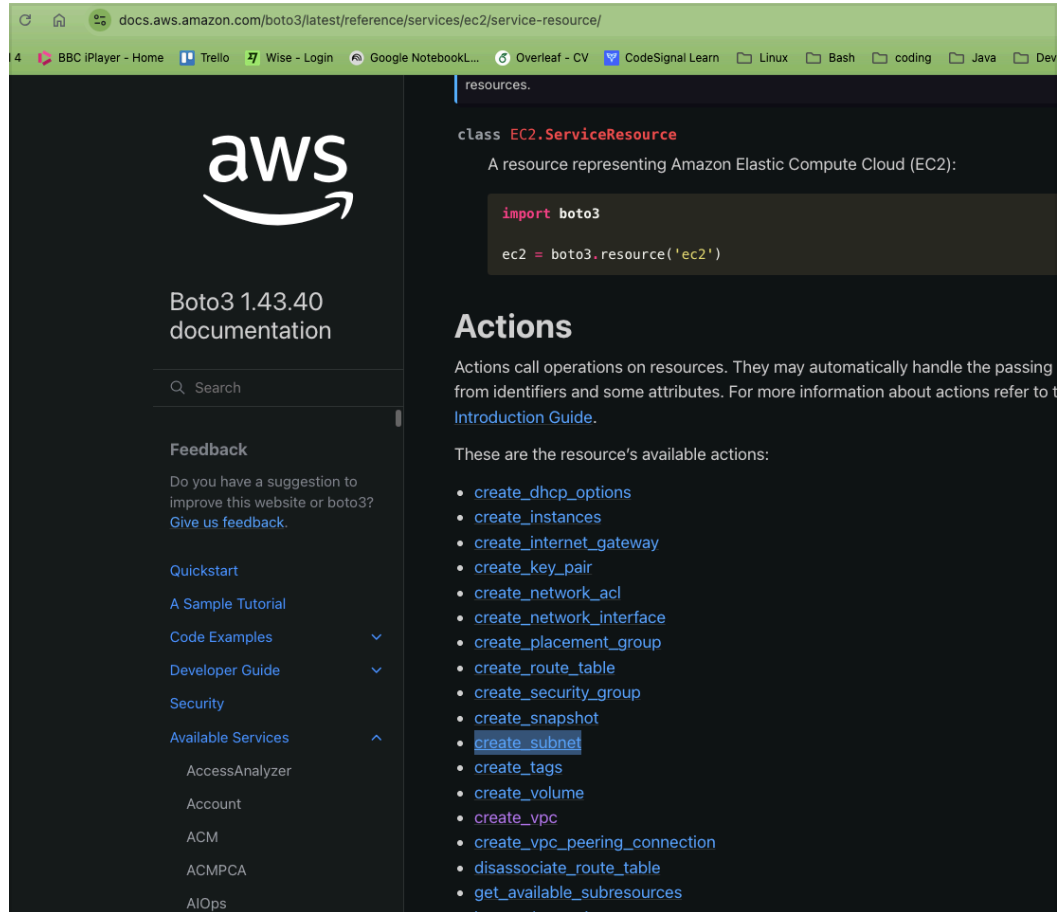


If we were using the client we will have to provide the vpc id and the subnet information. Because the client has all the vpcs we need to be specific of what

info from the client we want.

But with resources we get the specific object returned to us and we can work directly on that object.

Now, let's create 2 subnets and in the docs we find -> `create_subnet()`



The screenshot shows the AWS Boto3 documentation page for the EC2 ServiceResource. The page is titled "Boto3 1.43.40 documentation" and features the AWS logo. On the left, there is a sidebar with navigation links including "Feedback", "Quickstart", "A Sample Tutorial", "Code Examples", "Developer Guide", "Security", and "Available Services". The main content area is titled "Actions" and lists various actions available for the EC2 ServiceResource, such as `create_dhcp_options`, `create_instances`, `create_internet_gateway`, `create_key_pair`, `create_network_acl`, `create_network_interface`, `create_placement_group`, `create_route_table`, `create_security_group`, `create_snapshot`, `create_subnet`, `create_tags`, `create_volume`, `create_vpc`, `create_vpc_peering_connection`, `disassociate_route_table`, and `get_available_subresources`. The `create_subnet` action is highlighted in blue. Above the actions, there is a code snippet showing how to import boto3 and create an EC2 resource:

```
resources.  
  
class EC2.ServiceResource  
    A resource representing Amazon Elastic Compute Cloud (EC2):  
  
    import boto3  
  
    ec2 = boto3.resource('ec2')
```

So I create the 2 subnets

```

6     new_vpc = ec2_resource.create_vpc(
7         CidrBlock="10.0.0.0/16"
8     )
9
10    new_vpc.create_subnet(
11        CidrBlock="10.0.1.0/24"
12    )
13
14    new_vpc.create_subnet(
15        CidrBlock="10.0.2.0/24"
16    )

```

And let's add a name tag to our new vpc that once the vpc is created we will see in AWS console under the column name

Your VPCs (1) Info	
Search	
☐ Name	☐ VPC ID
☐ -	vpc-05e4d423ed8512e99

So service-resource has a function 'create_tags'

docs.aws.amazon.com/boto3/latest/reference/services/ec2/service-resource/

BBC iPlayer - Home Trello Wise - Login Google NotebookL... Overleaf - CV CodeSignal Learn Linux Bash coding Ja



Boto3 1.43.40 documentation

Search

Feedback

Do you have a suggestion to improve this website or boto3?
[Give us feedback.](#)

Quickstart

- [A Sample Tutorial](#)
- [Code Examples](#)
- [Developer Guide](#)
- [Security](#)
- [Available Services](#)

AccessAnalyzer

Actions

Actions call operations on resources. They may automatically handle the response from identifiers and some attributes. For more information about actions, see [Introduction Guide](#).

These are the resource's available actions:

- [create_dhcp_options](#)
- [create_instances](#)
- [create_internet_gateway](#)
- [create_key_pair](#)
- [create_network_acl](#)
- [create_network_interface](#)
- [create_placement_group](#)
- [create_route_table](#)
- [create_security_group](#)
- [create_snapshot](#)
- [create_subnet](#)
- [create_tags](#)
- [create_volume](#)
- [create_vpc](#)
- [create_vpc_peering_connection](#)
- [disassociate_route_table](#)
- [get_available_subresources](#)
- [import_key_pair](#)
- [register_image](#)

ServiceResource / Action / create_tags

create_tags

EC2.ServiceResource.create_tags(kwargs)**

Adds or overwrites only the specified tags for the specified Amazon EC2 resource or resources. When you specify an existing tag key, the value is overwritten with the new value. Each resource can have a maximum of 50 tags. Each tag consists of a key and optional value. Tag keys must be unique per resource.

For more information about tags, see [Tag your Amazon EC2 resources](#) in the *Amazon Elastic Compute Cloud User Guide*. For more information about creating IAM policies that control users' access to resources based on tags, see [Supported resource-level permissions for Amazon EC2 API actions](#) in the *Amazon Elastic Compute Cloud User Guide*.

See also: [AWS API Documentation](#)

Request Syntax

```
response = ec2.create_tags(
    DryRun=True|False,
    Resources=[
        'string',
    ],
    Tags=[
        {
            'Key': 'string',
            'Value': 'string'
        },
    ],
)
```

And I can copy what I need and use it in my code:

```

14     new_vpc.create_subnet(
15         CidrBlock="10.0.2.0/24"
16     )
17
18     new_vpc.create_tags(
19         Tags=[
20             {
21                 'Key': 'Name',
22                 'Value': 'my-vpc'
23             },
24         ]
25     )
26

```

I run my code too many times that I got an error

```

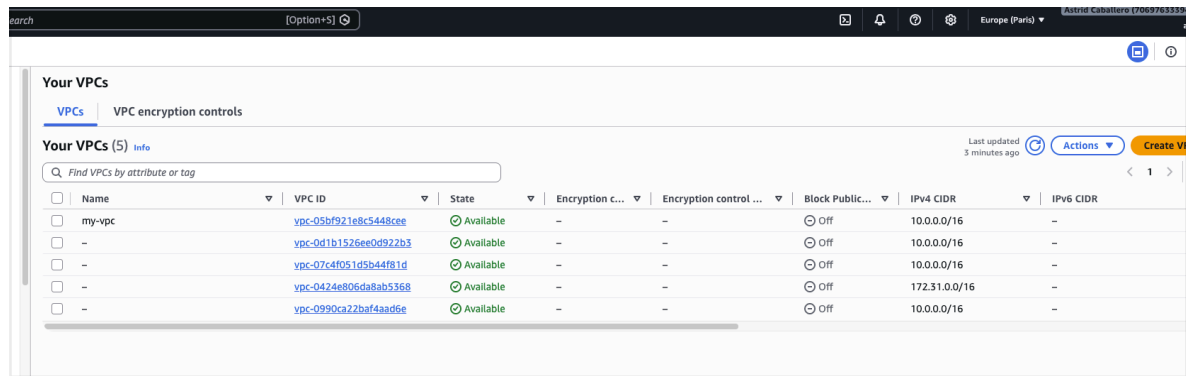
Run  main x
File "/Users/astrid/PythonProject/Automation_with_python/.venv/lib/python3.14/site-packages/boto3/responses.py", line 198, in _make_api_call
    response = getattr(parent.meta.client, operation_name)(*args, **params)
File "/Users/astrid/PythonProject/Automation_with_python/.venv/lib/python3.14/site-packages/botocore/client.py", line 553, in _make_api_call
    return self._make_api_call(operation_name, kwargs)
File "/Users/astrid/PythonProject/Automation_with_python/.venv/lib/python3.14/site-packages/botocore/client.py", line 981, in _make_api_call
    return func(*args, **kwargs)
File "/Users/astrid/PythonProject/Automation_with_python/.venv/lib/python3.14/site-packages/botocore/client.py", line 1035, in _raise_error_class
    raise error_class(parsed_response, operation_name)
botocore.exceptions.ClientError: An error occurred (VpcLimitExceeded) when calling the CreateVpc operation: The maximum number of VPCs has been reached.

Process finished with exit code 1

```

"raise error_class(parsed_response, operation_name)
botocore.exceptions.ClientError: An error occurred (VpcLimitExceeded) when calling the CreateVpc operation: The maximum number of VPCs has been reached."

I checked my AWS UI for region eu-west-3 -> Paris



And 'my-vpc' is there and the error tells me that I have reached the limit of vpcs that I can create.

Each run of the script created a new VPC before crashing on create_vpc, and default AWS accounts are capped at 5 VPCs per region.

I want to remove the vpcs and keep the default -> AI helps me to check the vpcs and it runs a batch command to list subnets, internet gateways, and NAT gateways for each non-default VPC:

```
-> for vpc in vpc-05bf921e8c5448cee vpc-0d1b1526ee0d922b3
vpc-07c4f051d5b44f81d vpc-0990ca22baf4aad6e; do
    echo "=== $vpc ==="
    echo "-- subnets --"
    aws ec2 describe-subnets --region eu-west-3 --filters "Name=vpc-
id,Values=$vpc" --query 'Subnets[].{SubnetId:SubnetId,Cidr:CidrBlock}' --
output table
    echo "-- internet gateways --"
    aws ec2 describe-internet-gateways --region eu-west-3 --filters
"Name=attachment.vpc-id,Values=$vpc" --query
'InternetGateways[].InternetGatewayId' --output text
    echo "-- nat gateways --"
    aws ec2 describe-nat-gateways --region eu-west-3 --filter "Name=vpc-
id,Values=$vpc" --query 'NatGateways[].NatGatewayId' --output text
    echo
done
```

And suggested

Easiest option — AWS Console:

1. Go to VPC → Your VPCs, select the checkbox for a non-default VPC (all except the 172.31.0.0/16 one — that's the AWS default, keep it).
2. Actions → Delete VPC.
3. The console shows you everything attached (subnets, route tables, etc.) and deletes them together in one confirmation step. If something else is attached (NAT gateway, EC2 instance, peering) it'll tell you and you'll need to remove that first.

From what I found before you stopped me:

- vpc-05bf921e8c5448cee (my-vpc): has 2 subnets (subnet-074d85db998d97089, subnet-08c28c69cd333af69), no IGW/NAT — console delete will handle it in one go.
- vpc-0d1b1526ee0d922b3: no subnets, no IGW, no NAT — deletes cleanly on its own.
- vpc-07c4f051d5b44f81d / vpc-0990ca22baf4aad6e: I hadn't finished checking these — worth confirming no subnets/NAT/IGW are attached before deleting (VPC → Your VPCs → click the VPC ID → check the Subnets/Internet Gateways tabs).

So I deleted the vpcs manually!